第01堂｜AI原生開發與LLM核心架構

學習目標

完成本堂課後，學員將能夠：

- 說明大語言模型（LLM）的基本原理與主要產業應用場景
- 描述 AI Agent 的系統架構與四大核心組成
- 體驗 Vibe Coding 的開發方式，並理解其長處與限制
- 認識 OpenClaw 平台的基礎概念
- 識別使用 LLM 所衍生的基本資安風險

核心概念說明

1. 大語言模型（Large Language Model，LLM）

大語言模型是一種以海量文字資料訓練而成的 AI 系統，能夠理解並生成自然語言。

一個簡單的比喻： 想像有一位助理，他從小閱讀了幾乎所有的書籍、文章、程式文件與對話紀錄。當你給他一個問題或指令，他會根據所有學過的知識生成一個最合理的回應。這就是 LLM 的運作方式。

常見的 LLM 產品：

產品	開發商	特點
GPT-4o	OpenAI	業界廣泛使用的通用模型
Claude	Anthropic	強調安全性與長文本處理
Gemini	Google	整合 Google 服務生態
Llama 3	Meta	開源模型，可在本地部署

LLM 的核心能力：

- **文字生成（Text Generation）**：根據輸入的 Prompt 產出文字內容
- **語意理解（Semantic Understanding）**：理解語境與隱含意圖，不只是字面意思
- **指令遵循（Instruction Following）**：根據具體指令執行特定任務
- **推理與分析（Reasoning）**：進行多步驟的邏輯推論

LLM 在產業的實際應用：

- 客服自動回覆：全天候回應客戶常見問題
- 文件摘要與分析：快速萃取大量報告的關鍵資訊
- 程式碼自動生成：加快軟體開發速度
- 行銷文案量產：針對不同受眾快速生成內容
- 資料分類與標記：自動整理非結構化資料

---

2. AI Agent 系統架構

AI Agent（AI 代理）是一種能夠**自主執行複雜任務**的 AI 系統。與普通的 LLM 對話不同，Agent 具有完整的「感知 → 思考 → 規劃 → 行動」迴圈。

直覺的比喻：

想像你雇用了一位助理，你告訴他：「幫我調查三個競爭對手的定價，整理成 Excel 報表後寄給我。」

- 助理會**自行規劃步驟**：搜尋網路 → 瀏覽各官網 → 紀錄資料 → 開啟 Excel → 填寫資料 → 寄送郵件
- 全程不需要你一步一步指示，他會自己完成

AI Agent 就是能夠**自主規劃並執行多步驟任務**的 AI 系統。

Agent 的四大組成要素：

```
          使用者任務指令
                │
                ▼
┌──────────────────────┐
│                      │
│   大腦（LLM）        │   ← 理解任務、制定決策
│                      │
└──────────────────────┘
            │
            ▼
┌──────────────────────┐
│                      │
│  規劃器（Planner）   │   ← 將複雜任務拆解為可執行步驟
│                      │
└──────────────────────┘
            │
            ▼
┌──────────────────────┐
│                      │
│   工具集（Tools）    │   ← 瀏覽器、程式執行器、API、檔案讀寫...
│                      │
└──────────────────────┘
            │
            ▼
┌──────────────────────┐
│                      │
│   記憶體（Memory）   │   ← 儲存過去的對話與執行紀錄
│                      │
└──────────────────────┘
```

---

3. Vibe Coding

Vibe Coding 是一種以**自然語言驅動**程式碼生成的開發方式。你不需要自己寫每一行程式語法，只需要用白話文描述你想要做的事，由 AI 工具（如 GitHub Copilot）幫你生成可執行的程式碼。

Vibe Coding 的長處：

- 大幅降低程式設計的學習門檻
- 快速將想法轉化為可執行的自動化工具
- 讓非技術背景的業務人員也能構建流程
- 加快原型（Prototype）開發速度

Vibe Coding 的限制：

- AI 生成的程式碼可能包含錯誤，**必須人工驗證**
- 對非常複雜或高安全需求的系統有一定侷限
- 若描述需求不清楚，AI 可能生成不符合期望的程式碼
- 不適合直接用於需要高精確度的核心系統

傳統開發 vs. Vibe Coding 比較：

	傳統開發	Vibe Coding
前置要求	熟悉程式語法與演算法	能清楚描述需求
開發速度	依工程師熟練度而定	快速產出原型
結果可靠性	工程師掌控	需人工驗證
適合場景	核心系統、高安全性	自動化工具、快速驗證

---

4. OpenClaw 平台簡介

OpenClaw 是目前產業廣泛採用的 AI Agent 視覺化開發平台，讓使用者以**拖拉積木**的方式組合 AI 工作流程，大幅降低 Agent 系統的開發難度。

OpenClaw 的主要特色：

- 直覺的視覺化流程編輯介面
- 支援串接多種 LLM 後端（GPT、Claude、Llama 等）
- 內建豐富的工具節點：網頁搜尋、讀取檔案、呼叫 API、資料庫操作等
- 支援多個 Agent 相互協作完成複雜任務

本課程後續堂次將會實際使用 OpenClaw 構建各種自動化 Agent 系統。

---

5. LLM 與 Agent 的資安風險

在學習 AI Agent 開發的同時，我們也必須了解其潛在的資安風險，才能在設計系統時做出正確的防護措施。

主要風險類型：

風險類型	說明	防護方向
Prompt Injection	惡意使用者透過精心設計的輸入，操控 AI 執行非預期的行為	對外部輸入進行驗證與過濾
資料洩漏	Agent 在執行任務時可能不慎將敏感資料傳送至外部 LLM	避免將機密資料送入外部 API

風險類型	說明	防護方向
過度授權	Agent 被賦予超出任務所需的系統權限	遵循「最小權限原則」
幻覺（Hallucination）	LLM 生成看似合理但實際錯誤的資訊	重要輸出需人工核實

> **重要觀念：** LLM 因為訓練方式的特性，有時會「自信地給出錯誤答案」。這在 Agent 系統中尤其危險，因為錯誤的決策會直接驅動工具執行行動。設計 Agent 時，務必在關鍵決策點加入人工確認機制。

實作步驟（課堂示範）

> 本堂課由教師示範操作，學員跟隨觀察並理解整體概念。

示範一：安裝開發環境

步驟 1：安裝 Python

1. 前往 https://www.python.org/downloads/ 下載最新版本
2. 安裝時勾選「Add Python to PATH」選項

步驟 2：安裝 Visual Studio Code

1. 前往 https://code.visualstudio.com/ 下載安裝
2. 完成後開啟 VS Code

步驟 3：安裝 GitHub Copilot

1. 在 VS Code 左側擴充套件搜尋欄輸入 `GitHub Copilot`
2. 點選安裝
3. 使用 GitHub 帳號登入並完成授權

步驟 4：測試 Copilot 正常運作

1. 點選左側 Copilot 圖示開啟 Chat 視窗
2. 輸入「你好，請用中文自我介紹」測試回應

示範二：用 Vibe Coding 生成第一個 AI 程式

步驟 1：建立工作資料夾 在桌面建立一個名為 `ai-course` 的資料夾，並用 VS Code 開啟（檔案 → 開啟資料夾）。

步驟 2：在 Copilot Chat 輸入需求 在 Copilot Chat 視窗輸入以下描述：

```
請幫我用 Python 寫一個程式，讓我在終端機輸入文字，
程式會呼叫 OpenAI 的 API，並把 AI 的回應顯示在螢幕上。
需要支援連續對話，輸入 "exit" 就結束程式。
請記得從環境變數讀取 API Key，不要把 Key 寫在程式碼裡。
```

步驟 3：觀察生成的程式碼結構

觀察 Copilot 生成的 Python 程式，注意理解以下部分：

- **API Key 的讀取方式**：為何不該把 Key 直接寫在程式碼裡？
- **對話歷史的儲存方式**：為何需要保存每一輪的對話？
- **請求與回應的流程**：程式如何把你的輸入傳給 AI 並接收回應？

步驟 4：加入工具調用功能

教師將進一步修改程式，加入一個「讀取/寫入檔案」的工具，讓這個程式能夠操作電腦上的文字檔案，示範 Agent 的基本概念：當 AI 有了「工具」，它就從一個「聊天機器人」升級為「可以做事的代理人」。

程式碼範例（虛擬碼）

基礎 LLM 對話程式

```
# === 初始化 ===
載入 openai 套件
設定 API_KEY = 從環境變數讀取("OPENAI_API_KEY")

# === 初始化對話歷史 ===
對話歷史 = []

# === 主要對話迴圈 ===
重複執行:
    讀取 使用者輸入

    如果 使用者輸入 == "exit":
        顯示 "再見！"
        結束程式

    將「使用者訊息」加入對話歷史

    呼叫 LLM API:
        傳入: [模型名稱, 對話歷史]
        接收: AI 回應文字

    將「AI 回應」加入對話歷史
    顯示 AI 回應
```

加入工具調用的基礎 Agent 機制

```
# === 定義可用工具 ===
工具A: 讀取檔案(檔案路徑) → 回傳檔案內文字
工具B: 寫入檔案(檔案路徑, 文字內容) → 儲存到檔案
工具清單 = [工具A, 工具B]

# === Agent 決策迴圈 ===
```

```
函式 Agent執行(任務描述):
    重複執行 直到任務完成:

        給 LLM 的輸入 = {
            系統指令: "你是一個能操作檔案的 AI 助理",
            任務: 任務描述,
            可用工具: 工具清單的名稱與說明,
            過去執行結果: 執行歷史
        }

        LLM 決策 = 呼叫 LLM(給 LLM 的輸入)

        如果 LLM 決定使用某個工具:
            執行該工具，取得結果
            將結果記錄到執行歷史
        否則(LLM 認為任務完成):
            回傳最終答案
```

練習題 / 作業

課堂思考練習

練習 1：找出 AI 可以幫你的事

列出你目前工作或日常生活中，**3 件可以讓 AI 幫你做的事**：

#	任務說明	目前需要多少時間	AI 能省下什麼？
1			
2			
3			

練習 2：AI 適用性判斷

針對以下情境，判斷是否適合讓 AI 自動完成，並說明理由：

情境	適合？	理由
撰寫一封客戶感謝信		
計算財務報表的精確數字		
整理 200 份問卷回饋的主題		
確認法律合約中的特定條款責任		

課後作業

在下堂課之前，請完成以下任務：

1. **安裝開發環境**：確保 VS Code 與 GitHub Copilot 已正常運作

2. **AI 任務實驗**：選一件你日常工作中的小任務，用 ChatGPT 或 Claude 幫你完成
3. **記錄觀察**，下堂課分享（2-3 分鐘）：
    - 你想完成什麼任務？
    - 你如何描述需求給 AI？（你輸入了什麼 Prompt？）
    - AI 給出了什麼回應？
    - 結果是否符合你的期望？哪裡還不夠好？

延伸閱讀

AI 基礎概念：

- OpenAI 官方文件 — https://platform.openai.com/docs
- GitHub Copilot 入門指南 — https://docs.github.com/en/copilot

AI Agent 深入了解：

- LangChain 官方文件 — https://docs.langchain.com（主流 Agent 開發框架）
- 《Building LLM-Powered Applications》— 實用的 LLM 應用開發指南

資訊安全：

- OWASP LLM Top 10 — LLM 應用十大安全風險清單（可搜尋「OWASP LLM Top 10」取得最新版）
- NIST AI 風險管理框架 — 企業導入 AI 的風險評估指引